Portfolio guidance

Your portfolio makes up half of your apprenticeship end-point assessment. It is documentation describing the work you have completed during your apprenticeship, and must be completed before the assessment gateway.

Assessment

The portfolio should describe real projects you've worked on as part of your job. It should demonstrate how you fulfil the KSBs.

Your portfolio will not be directly assessed. Instead it will be used as the basis of a 60 minute discussion with your assessor, who will ask questions based on the criteria. This format gives you a better chance to show the assessor what you know, since they can ask questions about things that were not fully covered in the portfolio.

Evidence

Each piece of evidence in your portfolio should be a report describing some work you completed for a project. It is expected that each of these entries will cover multiple KSBs.

For example instead of just describing some tests you wrote for a feature you would describe the entire implementation of the feature, allowing you to demonstrate the testing criteria and several others.

You can think of each entry in your portfolio as a small project write-up. It should describe what you did and include "supporting evidence" like code samples, user stories, work tickets, documentation or other pieces of work, progress reviews, colleague/client feedback, and witness testimonies.

It is suggested that you write evidence and then map to the KSBs after you have finished writing up. This helps to avoid responding directly to KSBs. There's a balance to strike between writing a narrative that describes what you've been working on, and writing documentation relevant to the KSBs.

Valid evidence

All the work you include must have been completed for your employer. You cannot include side projects or other non-work activity.

Everything you include must be work that you did. It is okay for the project to have other team members, as long as you are primarily describing work you completed. For example if you implemented a feature but a colleague wrote all the tests for it you would not be able to include the tests as evidence to fulfill the testing criteria.

The evidence should be directly relevant to the KSBs; there is no point describing some work if you can't map it to any of the KSBs. You will be required to submit a mapping document that connects each KSB to the corresponding evidence.

Good evidence

Try to only use "I". The assessor is not interested in what "we" did—they need things to be directly attributable to you.

Your evidence should cover "what", "how", "with whom" and "why". For example: What did you do? How did you do it? How did your role fit into the wider team? Why was the work necessary? Why did you choose the approach you took?

Writing the portfolio

The portfolio "typically contains 10 pieces of evidence". This is a rough measure of how much to include and there's no exact measure for how much you need to write. So far, we've seen portfolios in the range of 5,000 - 15,000 words; there isn't a minimum or maximum word count.

It is important that you have covered every single KSB, at least once. We'd recommend trying to cover each one twice (or more) with your evidence.

For example there are 24 KSBs, so assuming you cover 6 unique ones in each entry you will only need to write about 4 projects.

There is no template for evidence and you might have a different structure for each piece. For example, in one piece of evidence you may have a heading for each stage of the Software Development lifecycle and give detail on how a whole feature was built; in another you may focus only on testing and give detail on different methodologies, frameworks and why you chose to use those.

Portfolio KSBs

The apprenticeship standard defines certain knowledge, skills and behaviours that are required for the occupation. An apprentice demonstrates these through their portfolio.

Filter

All

Knowledge

Skills

Behaviours

K1

all stages of the software development life-cycle (what each stage contains, including the inputs and outputs)

K3

the roles and responsibilities of the project life-cycle within your organisation, and your role

K4

how best to communicate using the different communication methods and how to adapt appropriately to different audiences

K5

the similarities and differences between different software development methodologies, such as agile and waterfall.

K7

software design approaches and patterns, to identify reusable solutions to commonly occurring problems

K8

organisational policies and procedures relating to the tasks being undertaken, and when to follow them. For example the storage and treatment of GDPR sensitive data.

K10

principles and uses of relational and non-relational databases

K12

software testing frameworks and methodologies

S2

develop effective user interfaces

S3

link code to data sets

S5

conduct a range of test types, such as Integration, System, User Acceptance, Non-Functional, Performance and Security testing.

S8

create simple software designs to effectively communicate understanding of the program

S9

create analysis artefacts, such as use cases and/or user stories

S13

follow testing frameworks and methodologies

S14

follow company, team or client approaches to continuous integration, version and source control

S15

communicate software solutions and ideas to technical and non-technical stakeholders

S17

interpret and implement a given design whist remaining compliant with security and maintainability requirements

B1

Works independently and takes responsibility. For example, has a disciplined and responsible approach to risk and stays motivated and committed when facing challenges

B4

Works collaboratively with a wide range of people in different roles, internally and externally, with a positive attitude to inclusion & diversity

B5

Acts with integrity with respect to ethical, legal and regulatory ensuring the protection of personal data, safety and security.

B6

Shows initiative and takes responsibility for solving problems within their own remit, being resourceful when faced with a problem to solve.

B7

Communicates effectively in a variety of situations to both a technical and non-technical audience.

B8

Shows curiosity to the business context in which the solution will be used, displaying an inquisitive approach to solving the problem. This includes the curiosity to explore new opportunities, techniques and the tenacity to improve methods and maximise performance of the solution and creativity in their approach to solutions.

B9

Committed to continued professional development.

Portfolio mapping

Your portfolio maps to a set of KSBs and your documentation should include evidence which covers each one of these. To aid the assessors in identifying where you've met each KSB, you'll submit a documentation which maps the KSBs to sections of your evidence.

Your portfolio demonstrates your understanding and competancy in a set of Knowledge, Skills and Behaviours.

You will map KSBs to your evidence to ensure you've covered every one of these and to help the assessor identify what you might talk about in your professional discussion. This mapping document is submitted alongside your portfolio when you enter Gateway.

In this document, list every Portfolio KSB and document where in your evidence you've met that KSB with reference to page or section numbers.

Example of mapping document:

Evidence Documents

ID	Title	Link
1	Work project	

Knowledge Requirements

ID	Description	Evidence
K1	All stages of the software development life-cycle	Work project, section 1.1 - 1.8
K3	The roles and responsibilities of the project life-cycle within your organisation, and your role	
K4	How best to communicate using the different communication methods and how to adapt appropriately to different audiences	
K5	The similarities and differences between different software development methodologies, such as agile and waterfall.	
K7	Software design approaches and patterns, to identify reusable solutions to commonly occurring problems	
K8	Organisational policies and procedures relating to the tasks being undertaken, and when to follow them. For example, the storage and treatment of GDPR sensitive data.	
K10	Principles and uses of relational and non-relational databases	

K12 Software testing frameworks and methodologies

Skill Requirements

ID Description Evidence

S2 Develop effective user interfaces

S3 Link code to data sets

S5 Conduct a range of test types, such as Integration, System, User Acceptance, Non-Functional, Performance and Security testing.

S8 Create simple software designs to effectively communicate understanding of the program

S9 Create analysis artefacts, such as use cases and/or user stories

S13 Follow testing frameworks and methodologies

S14 Follow company, team or client approaches to continuous integration, version and source control

S15 Communicate software solutions and ideas to technical and non-technical stakeholders

S17 Interpret and implement a given design whist remaining compliant with security and maintainability requirements

Behaviour Requirements

ID Description Evidence

B1 Works independently and takes responsibility. For example, has a disciplined and responsible approach to risk and stays motivated and committed when facing challenges

B4 Works collaboratively with a wide range of people in different roles, internally and externally, with a positive attitude to inclusion & diversity

B5 Acts with integrity with respect to ethical, legal and regulatory ensuring the protection of personal data, safety and security.

B6 Shows initiative and takes responsibility for solving problems within their own remit, being resourceful when faced with a problem to solve.

B7 Communicates effectively in a variety of situations to both a technical and non-technical audience.

B8 Shows curiosity to the business context in which the solution will be used, displaying an inquisitive approach to solving the problem. This includes the curiosity to explore new opportunities, techniques and the tenacity to improve methods and maximise performance of the solution and creativity in their approach to solutions.

B9 Committed to continued professional development.

Portfolio criteria

Your portfolio makes up half of your apprenticeship endpoint assessment. It is a document describing the work you have completed during your apprenticeship, and must be completed before the assessment gateway period.

Assessment criteria

There are 21 assessment criteria required to gain a "Pass" grade. There are an additional 3 criteria to attain a "Distinction" grade. However the Distinction criteria overlap with 3 of the Pass criteria, so there are functionally 20 criteria to meet.

You must use evidence to demonstrate that you meet each of the criteria. For example to meet the criterion "Describes basic software testing frameworks and methodologies" you would need to describe how you wrote tests for a project you worked on.

Here are the criteria:

1. Describes all stages of the software development lifecycle. K1
2. Describes the roles and responsibilities of the project lifecycle within their organisation, and their role. K3
3. Describes methods of communicating with all stakeholders that is determined by the audience and/or their level of technical knowledge.

   Distinction: Compares and contrasts the different types of communication used for technical and non-technical audiences and the benefits of these types of communication methods. K4, S15, B7
4. Describes the similarities and differences between different software development methodologies, such as agile and waterfall. K5
5. Suggests and applies different software design approaches and patterns, to identify reusable solutions to commonly occurring problems (include Bespoke or off-the-shelf).

   Distinction: Evaluates and recommends approaches to using reusable solutions to common problems. K7
6. Explains the relevance of organisational policies and procedures relating to the tasks being undertaken, and when to follow them including how they have followed company, team or client approaches to continuous integration, version, and source control. K8, S14
7. Applies the principles and uses of relational and non-relational databases to software development tasks. K10
8. Describes basic software testing frameworks and methodologies.

Distinction: Evaluates the use of various software testing frameworks and methodologies and justifies their choice. K12

9. Explains, their own approach to development of user interfaces. S2
10. Explains, how they have linked code to data sets. S3
11. Illustrates how to conduct test types, including Integration, System, User Acceptance, Non-Functional, Performance and Security testing including how they have followed testing frameworks and methodologies. S5, S13
12. Creates simple software designs to communicate understanding of the programme to stakeholders and users of the programme. S8
13. Creates analysis artefacts, such as use cases and/or user stories to enable effective delivery of software activities. S9
14. Explains, how they have interpreted and implemented a given design whilst remaining compliant with security and maintainability requirements. S17
15. Describes, how they have operated independently to complete tasks to given deadlines which reflect the level of responsibility assigned to them by the organisation'. B1
16. Illustrates how they have worked collaboratively with people in different roles, internally and externally, which show a positive attitude to inclusion & diversity. B4
17. Explains how they have established an approach in the workplace which reflects integrity with respect to ethical, legal, and regulatory matters and ensures the protection of personal data, safety and security. B5
18. Illustrates their approach to meeting unexpected minor changes at work and outlines their approach to delivering within their remit using their initiative. B6
19. Explains how they have communicated effectively in a variety of situations to both a technical and non-technical audience. B7
20. Illustrates how they have responded to the business context with curiosity to explore new opportunities and techniques with tenacity to improve solution performance, establishing an approach to methods and solutions which reflects a determination to succeed. B8
21. Explains how they reflect on their continued professional development and act independently to seek out new opportunities. B9

Portfolio examples

Example portfolio evidence

Solent Mind

This is an early example created by FAC to give direction to our first cohort of apprentices approaching EPA. The project used was one of FAC22's Tech for Better projects. This is purely a demonstration of reasonable evidence and not a template for all evidence to follow.

Introduction

This piece of evidence documents the work completed by FAC22 on Tech for Better. Artefacts (including code snippets and screenshots) are taken from Solent Mind. This project was created for an external client, and can be considered built as part of the work of Founders and Coders as an organisation.

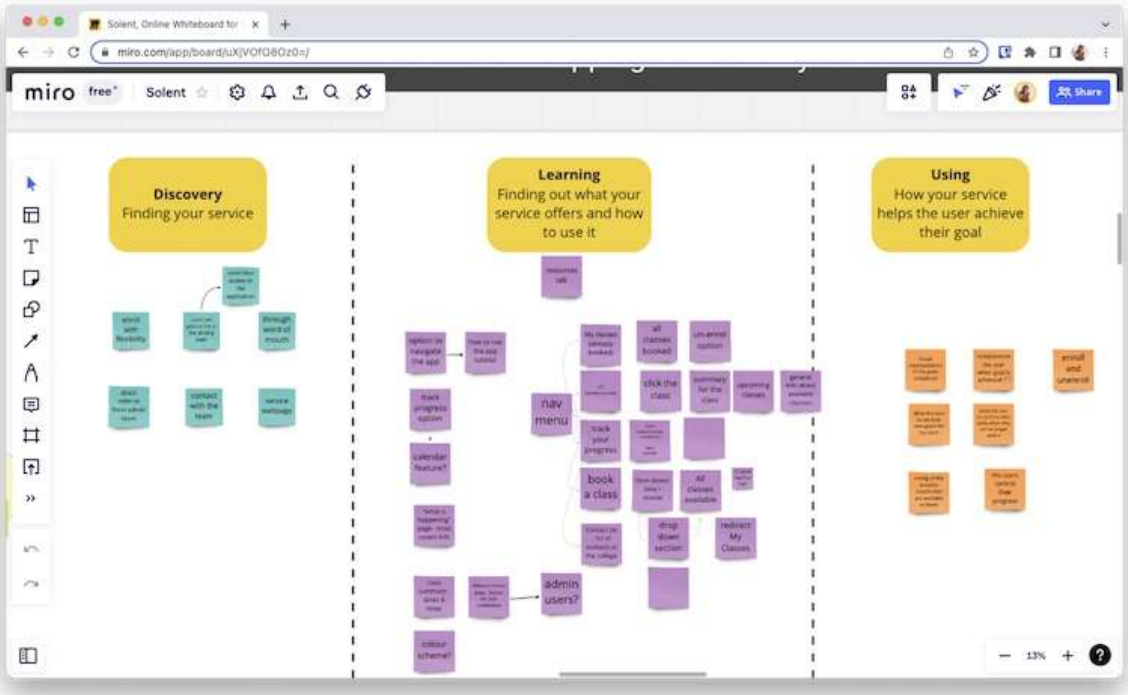
Software Development Cycle

For this project, I was part of a team who worked in an agile way. I followed six of the seven stages of the software development cycle.

Rather than deciding exactly on what we'd build straight away, my team took feedback from our users to iterate on our prototype before making a final decision on our tech stack. I was involved in sprint planning at the start of each period of work, and reviewed my progress at the end of the sprints.

Planning

With the client, I worked through exercises in a Discovery workshop. Completing a Problem Statement, User Personas and preparing for user research meant my client had a good idea of their users' needs. I then planned a User Journey using Miro, an online whiteboarding tool, which details the steps the user will complete when using the application.



An image of our user journey planned on Miro

During this project, I considered how the team could build our product in a way which could be repeated for other projects, or made use of existing tools to maximise what could be accomplished in a short period of time. These included:

Organizing architecture for a bigger in scale product and managing tasks within a larger team
Code modularization and code reusability
Learning to use Supabase with Next.js for the backend
Using states more efficiently and only when needed
Learning Tailwind CSS
understanding how CSS customization works with the use of a framework
adapting the framework to our needs
Debugging code, tracing code paths and refactoring code for optimization purposes
Analysis
I completed a code planning exercise in consulation with my technical mentor during the Design sprint. Here, I went into depth on the software requirements and made notes on how the stack would be affected.

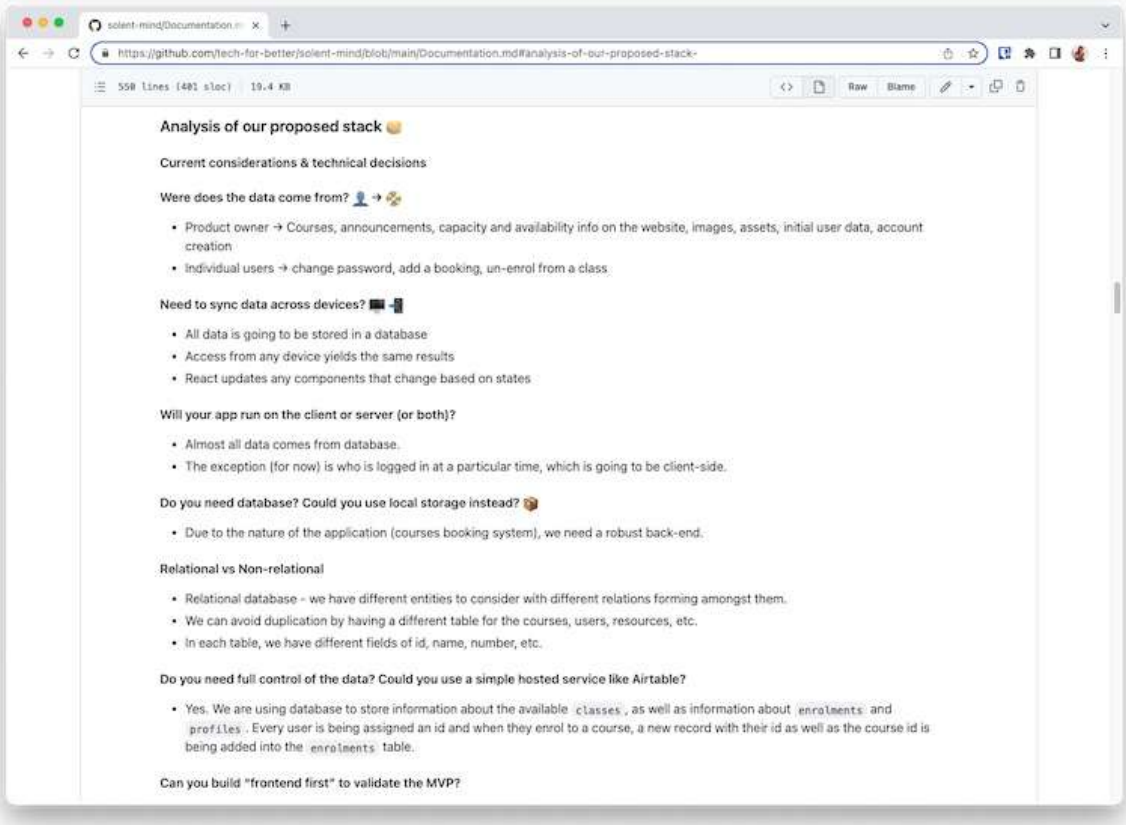
In code planning I considered:

Who the data would come from, and how I'd approach different paths based on this

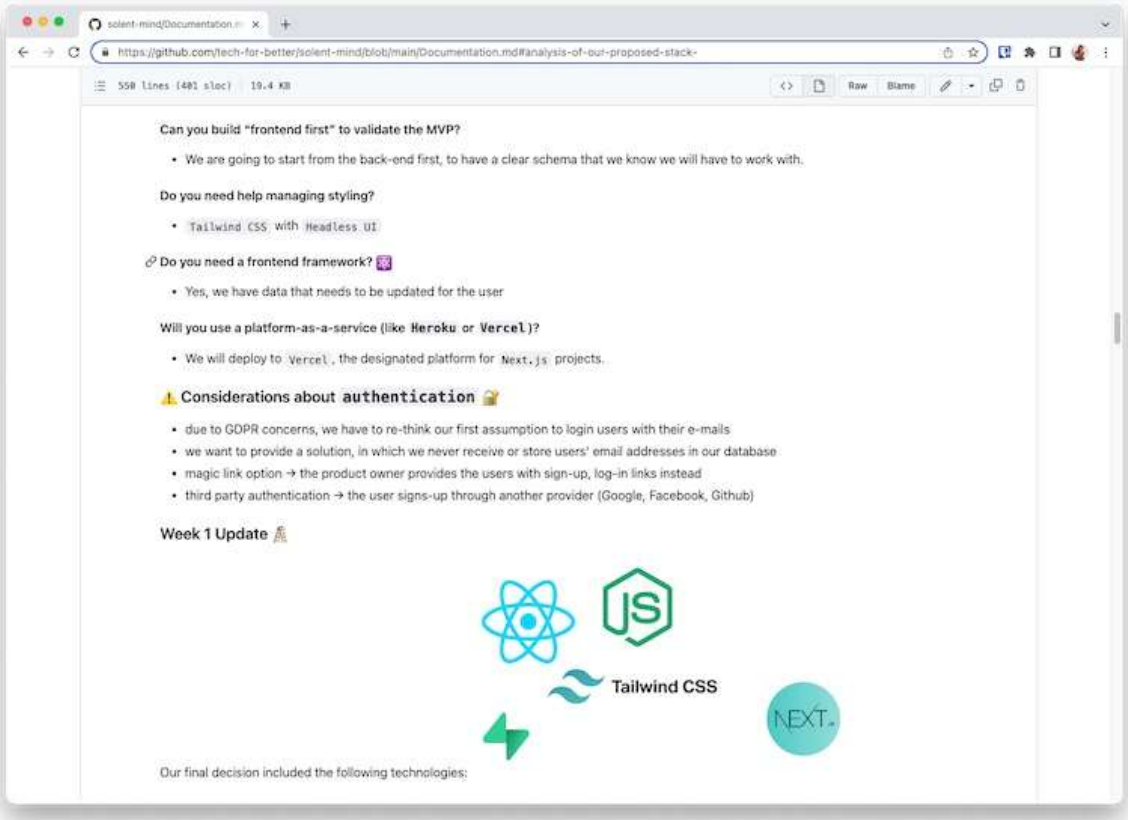
How data could be synchronised across devices. I opted to use a database to store data so that the app would present the same data for a user on any device.

That I wanted to build a full-stack app which could store data in a database and that any user would need to be authenticated to access their account

How I could manage styling in a component based application. I recommended that the team use Tailwind CSS which allowed us to quickly and consistently style pages of the application.



Analysis documentation shows technical considerations



Second analysis document shows further technical considerations

Design

I tried to make as many reusable elements as possible and keep an even design on all pages.

The main characteristics are: containers with softly rounded corners, bolder borders with soft glow and a rounder font as well.

The main elements on each page are placed in containers, whereas secondary information is left plain.

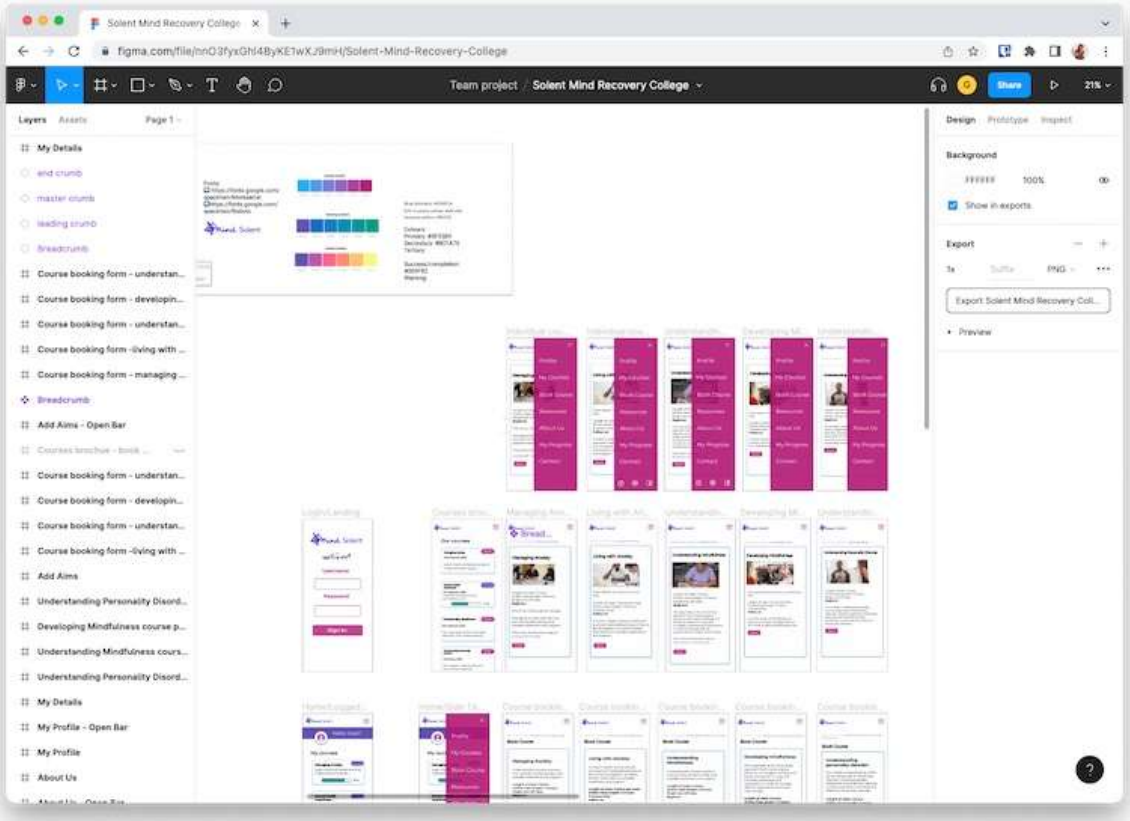
Resources


mind Solent
Resources
Here you can find articles curated by our medical team.
type your search
anxiety
depression
panick attacks
education
social life
Suggested articles
Managing stressful situations
6 min read
anxiety
depression
panic attacks
Source: www.nhs.uk
If you regularly make time for fun and relaxation, you'll be in a better place to handle life's stressors.
Read more...
How seasons may affect your mood
4 min read
anxiety
depression
Source: www.nhs.uk
If you regularly make time for fun and relaxation, you'll be in a better place to handle life's stressors.
Read more...
Prepare for stress-free exams
9 min read
anxiety
education
Source: www.nhs.uk
If you regularly make time for fun and relaxation, you'll be in a better place to handle life's stressors.
Read more...

Contact form

Screenshot of resources page Screenshot of contact page

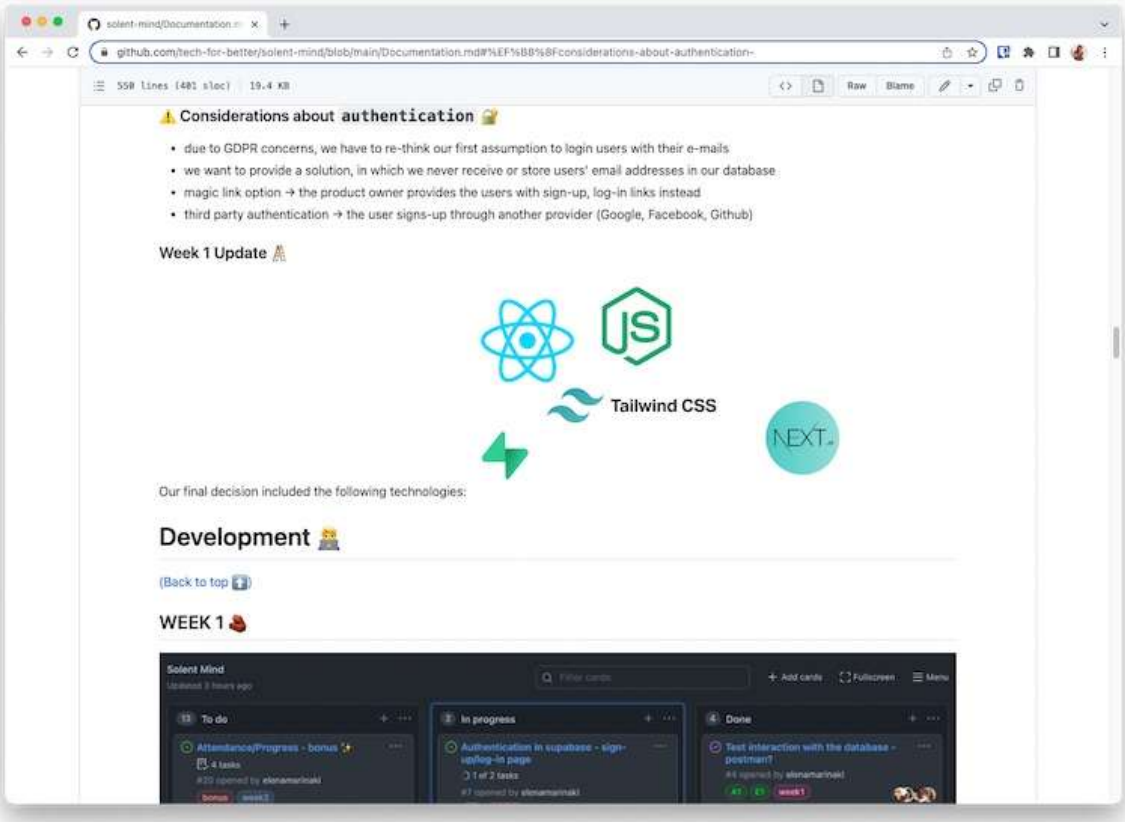
The first week was dedicated to designing and prototyping, including wireframing all main pages of the application, so I could test it with users before the build sprint began. My team used Miro and Figma for collaborative brainstorming and design.



Figma screenshot showing a mock-up of the application

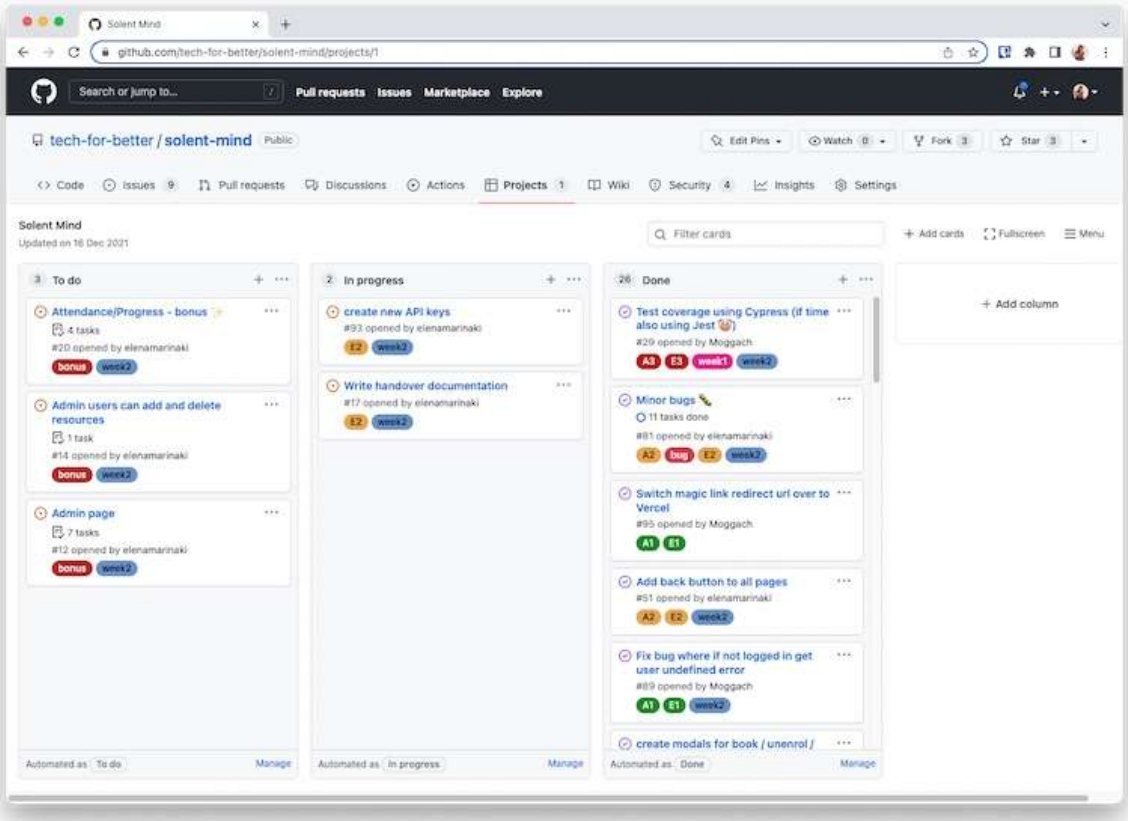
Building

I considered the implications of storing user email addresses in a self-managed database. Instead, I opted for a solution that relies on OAuth login managed by third-party providers like Google.



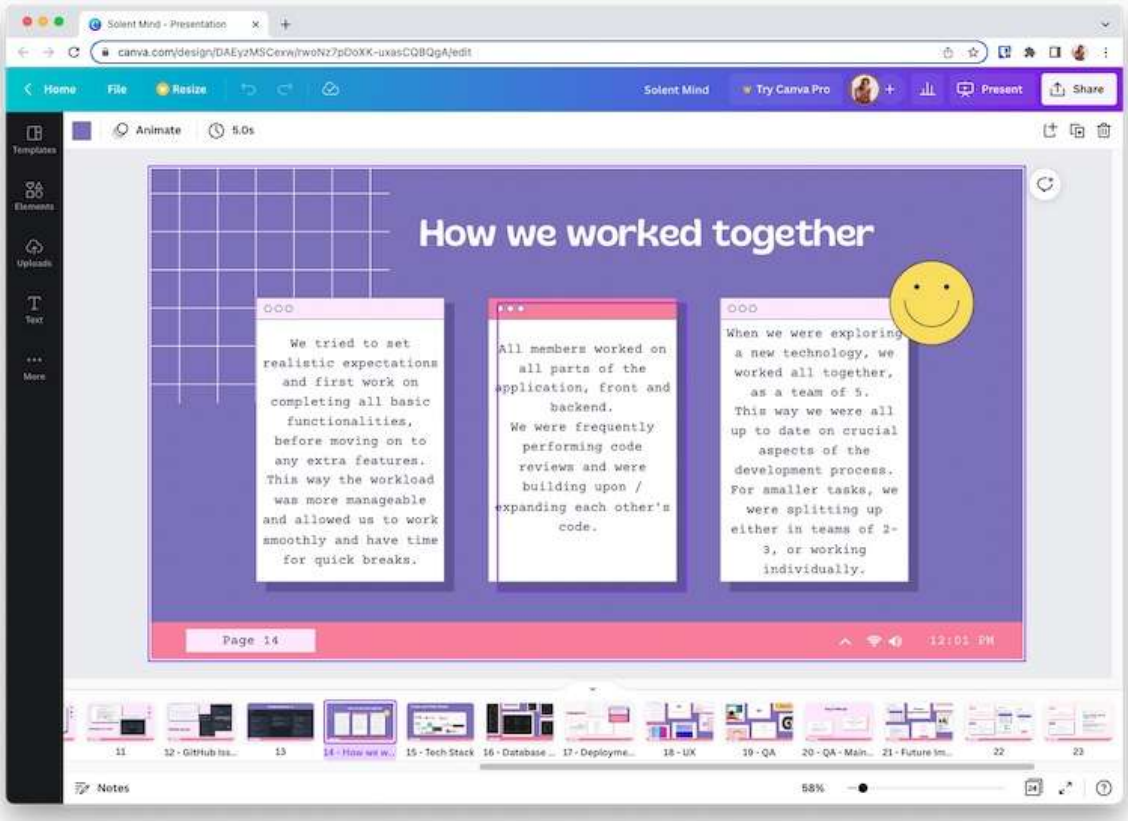
documentation showing authentication and security considerations

Our team delegated tasks on GitHub and used the Assignee feature on each issue to mark who was working on the specific task.



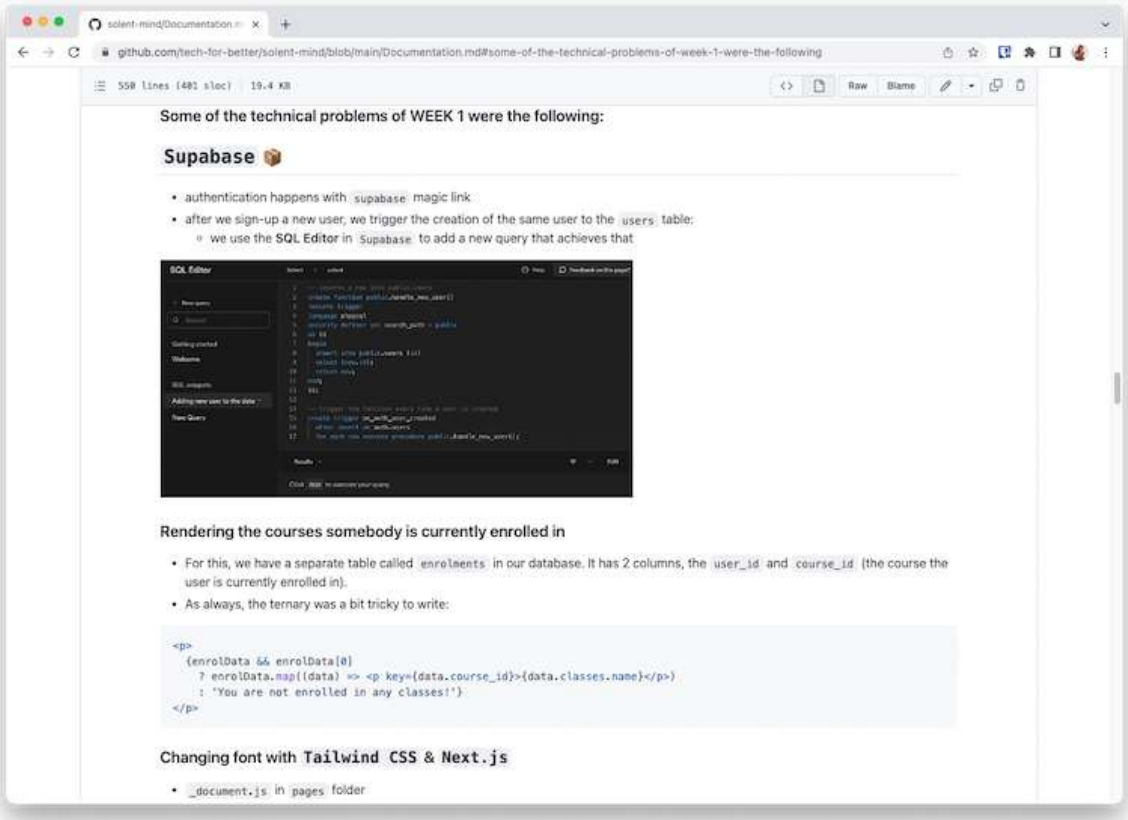
Screenshot of a kanban board showing project tasks

Each member of our team had a specific role in the development cycle and we collaborated to ensure all our responsibilities were fulfilled. I led on Quality Assurance which included delegating test writing to members of the team; ensuring that code was written consistently by different team members; and configuring a linter for all team members to use.



Presentation slide describes how we worked together

My team encountered some issues when setting up Supabase and I worked to resolve these in the first week.



Some of the technical problems of WEEK 1 were the following:

Supabase

- authentication happens with `supabase` magic link
- after we sign-up a new user, we trigger the creation of the same user to the `users` table:
  - we use the **SQL Editor** in `Supabase` to add a new query that achieves that

Rendering the courses somebody is currently enrolled in

- For this, we have a separate table called `enrolments` in our database. It has 2 columns, the `user_id` and `course_id` (the course the user is currently enrolled in).
- As always, the ternary was a bit tricky to write:

```
<p>
  {enrolData && enrolData[0]
    ? enrolData.map((data) => <p key={data.course_id}>{data.classes.name}</p>)
    : 'You are not enrolled in any classes!'}
</p>
```

Changing font with `Tailwind CSS` & `Next.js`

- `_document.js` in `pages` folder

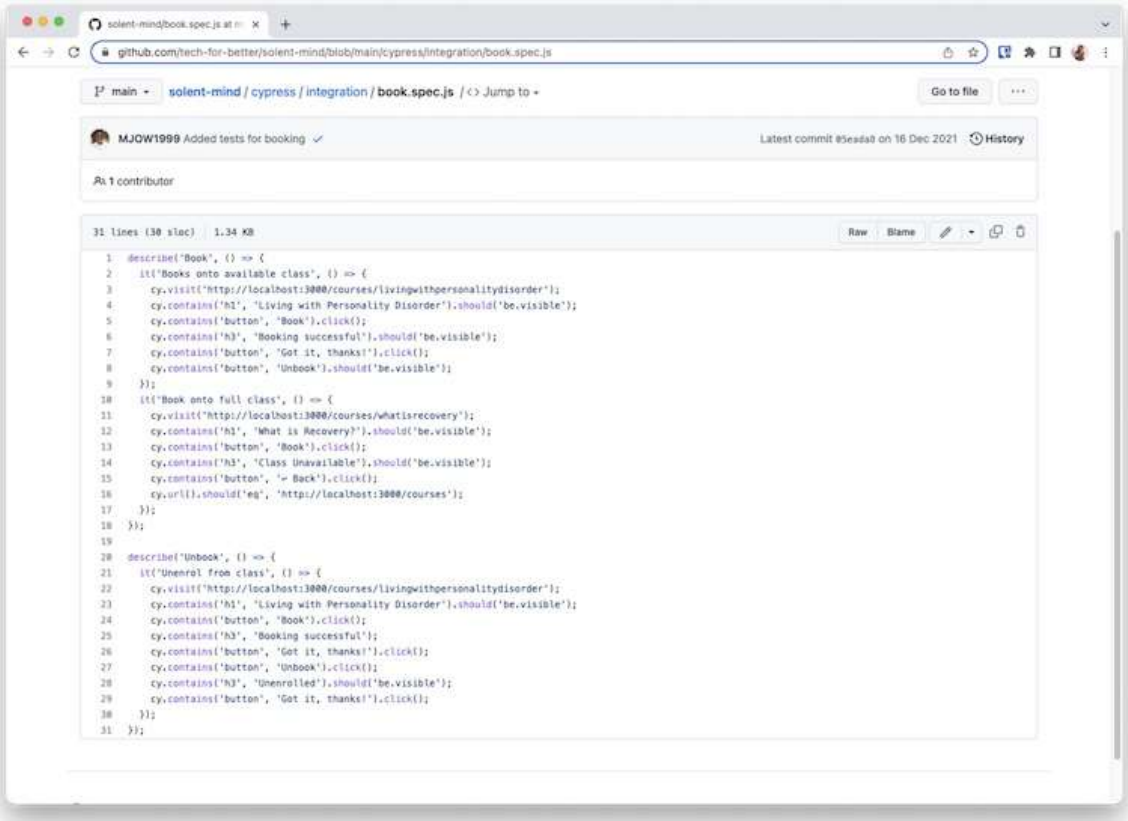
Documentation of Supabase issues

Testing

On this project, I looked into end-to-end testing, which checks the app behaves as expected when I emulate tasks a user would complete. As well, I considered using unit tests to check that individual parts of my application were functioning correctly.

I focused on end-to-end testing the application using Cypress. My tests closely matched the experience of a user. In the example below, you will see that I check the /livingwithpersonalitydisorder initially contains a heading and a button. When the user clicks the button which is labelled 'Book', the page renders again. The user should then see the heading 'Booking successful', and two buttons.

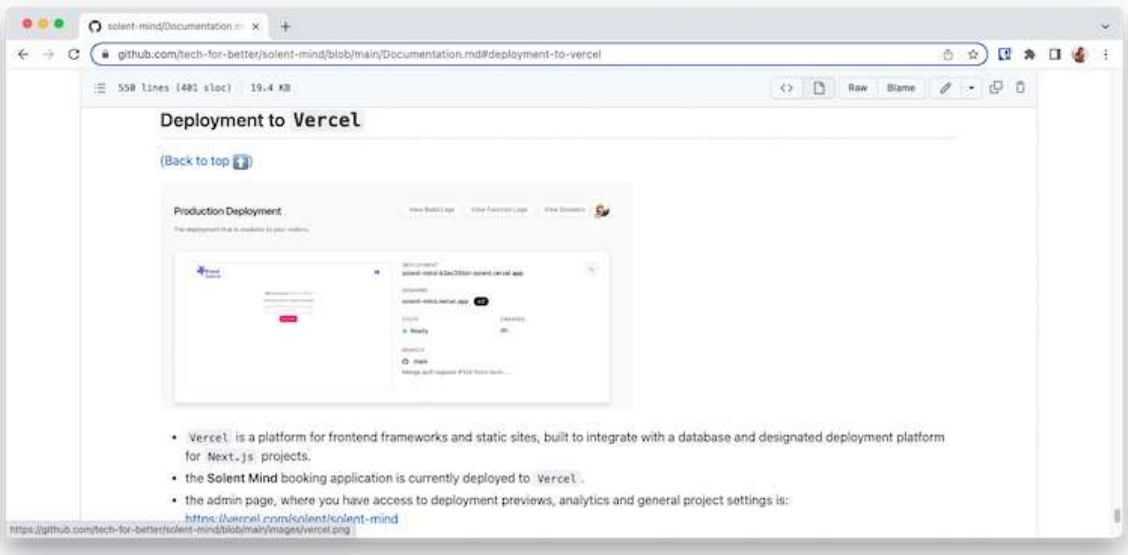
I was working to tight time constraints and I communicated to my team that I felt we should not focus on unit testing. I decided unit testing would not provide as much value for the effort writing them and it would not give as much insight as testing holistically using Cypress.



Screenshot of the code for a Cypress test

Deployment

I used the Vercel hosting platform for deployment, because it is developed specifically for the framework I used to build the application. It has a robust free tier and is relatively simple to work with, which was important in handing over the project to the non-technical Product Owner.



Documentation showing Deployment considerations

Maintenance

This is the only stage which I did not focus on for this project. The app will not be maintained by my team, but has been handing to the Product Owner with relevant documentation.

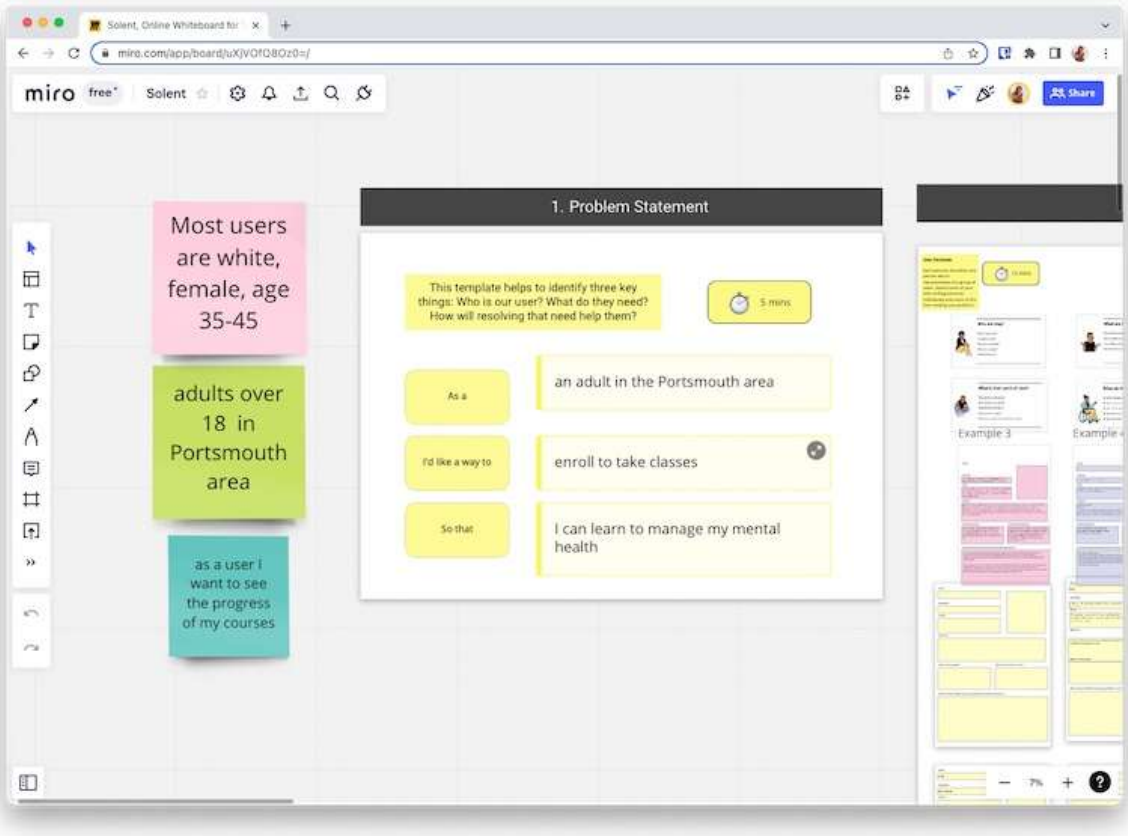
Sprint Planning

@todo

Working with a Client

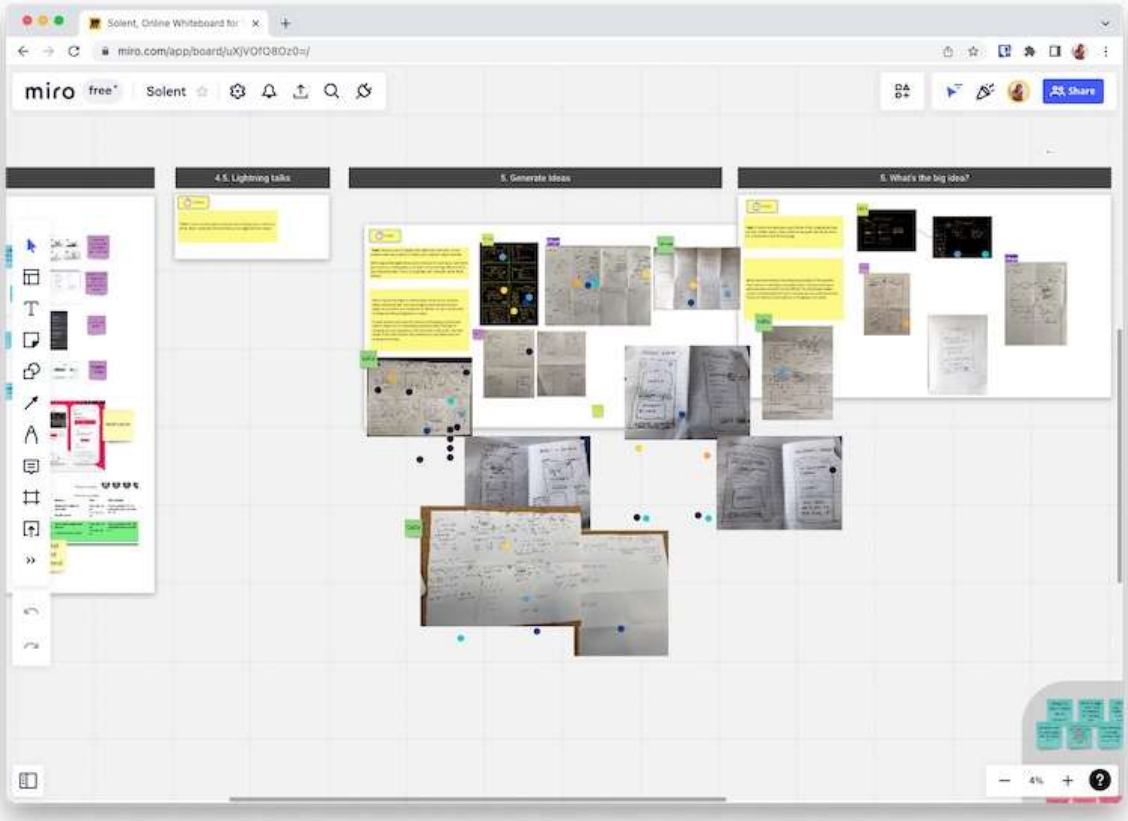
The Product Owner

My Product Owner came to the team with a statement of the problem their users are facing, and I worked with them to understand this and consider how a piece of software could respond to the needs of their users.



Screenshot of the problem statement: as an adult in the Portsmouth area, I'd like a way to enrol to take classes, so that I can learn to manage my mental health

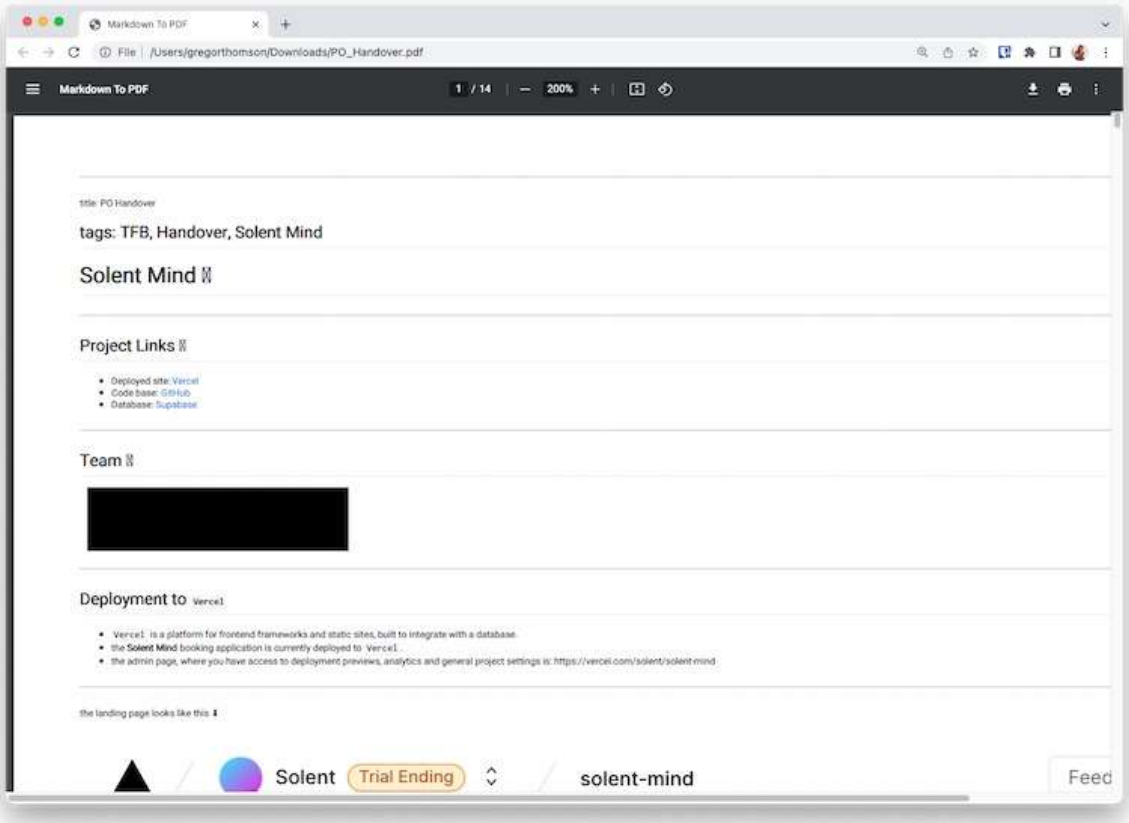
The design process involved my Product Owner and guiding them through the design process, including creating low fidelity wireframes on paper and Miro.



Paper wireframes each show different ideas of what the app will look like

I had a number of conversations with my Product Owner throughout the build to check in and keep them up to date with our progress. And they were involved in Sprint Planning and Reviews.

On completing the project, I wrote documentation for the Product Owner which included the details needed to maintain the Product beyond the scope of our engagement.

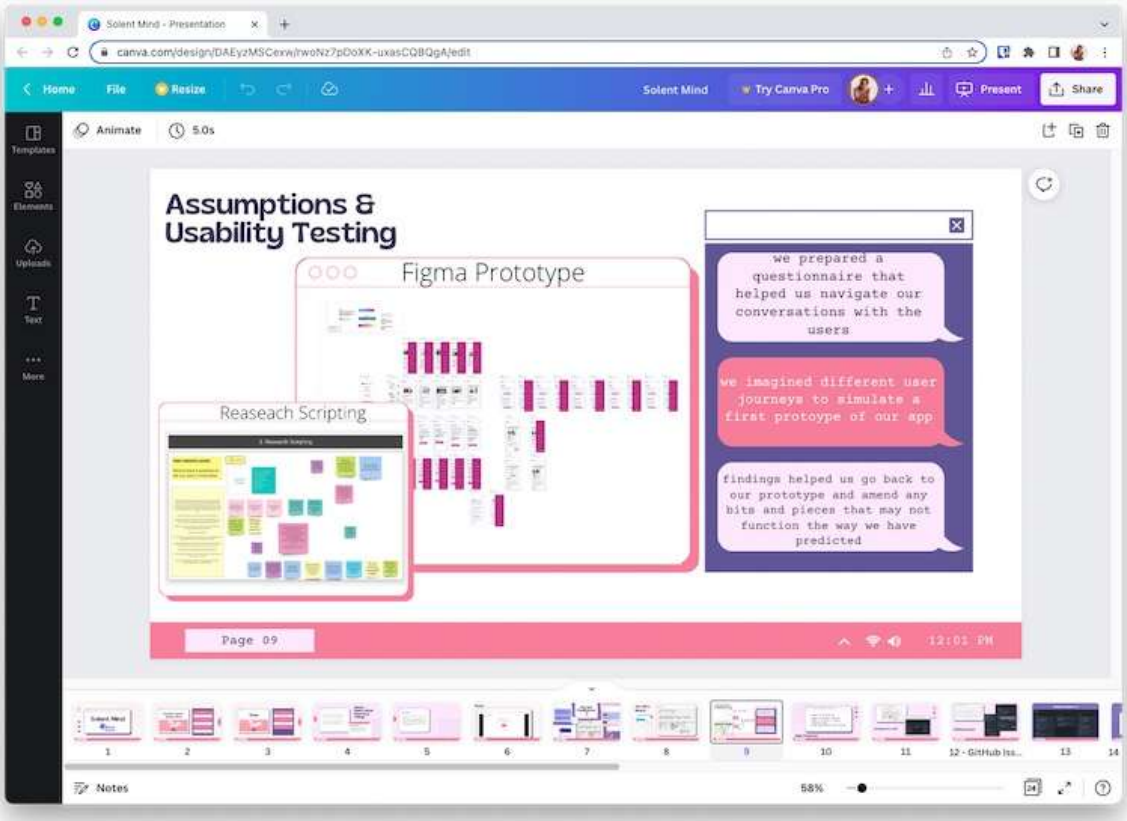


Handover documentation for the Product Owner

The user group

During Usability Testing, I communicated directly with our Product Owner's user group. This included writing up an introduction to the Product and how I'd be conducting the testing.

In these conversations, I kept the technical focus light and asked users to give us feedback on the prototype.

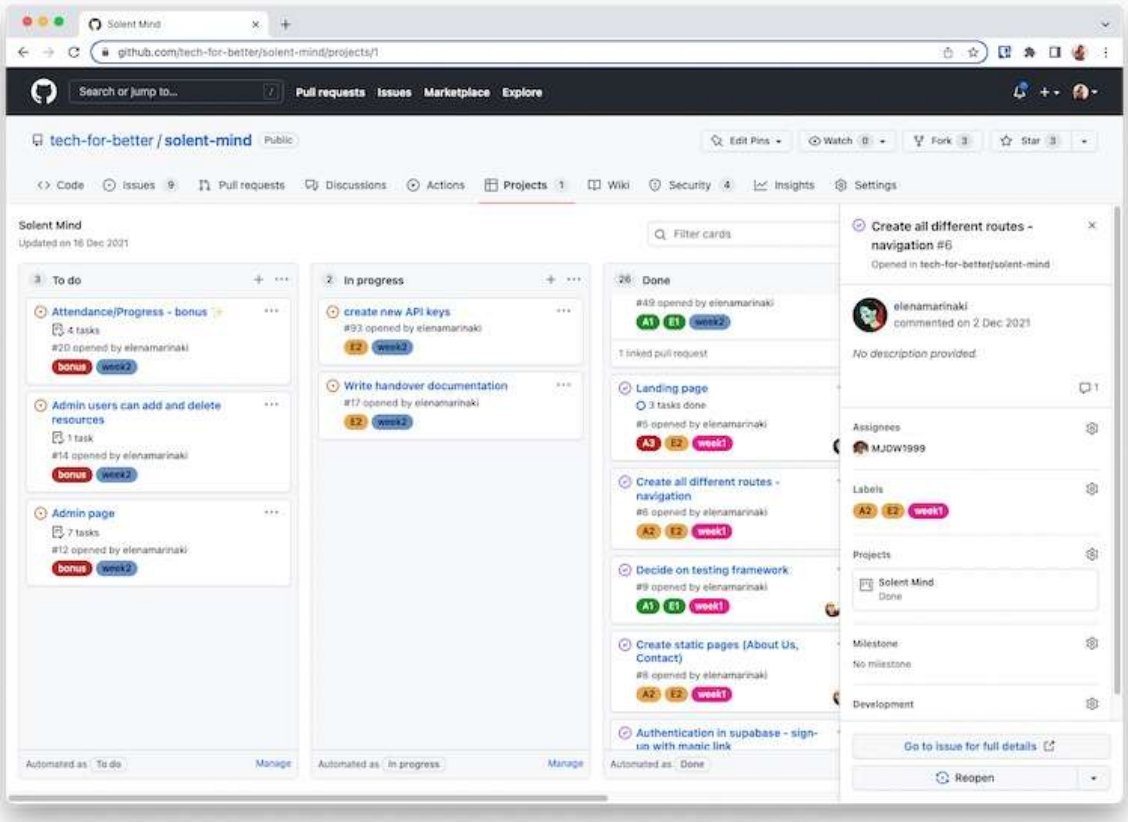


Presentation slide discusses how we conducted Usability Testing

Contrasting to these, were conversations with my technical mentors. I had a number of project scoping calls and conversations around my project role. These focused on the technical aspects of the MVP and involved discussing the specific technologies, and app architecture the team would use.

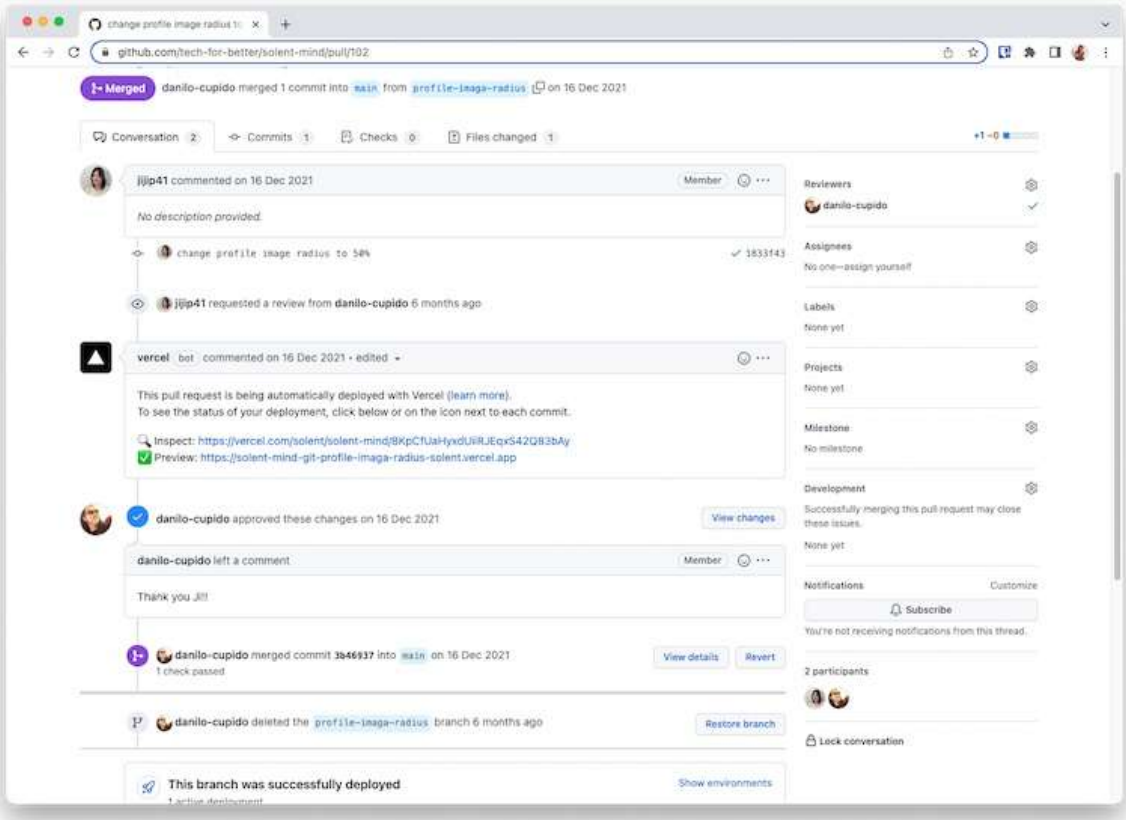
The company - FAC

The team managed sprints using GitHub's project board and our Scrum Facilitator led a daily stand-up meeting. My employer, Founders and Coders use the Scrum development method and this is what I followed in my project.



Project board shows a task to be completed

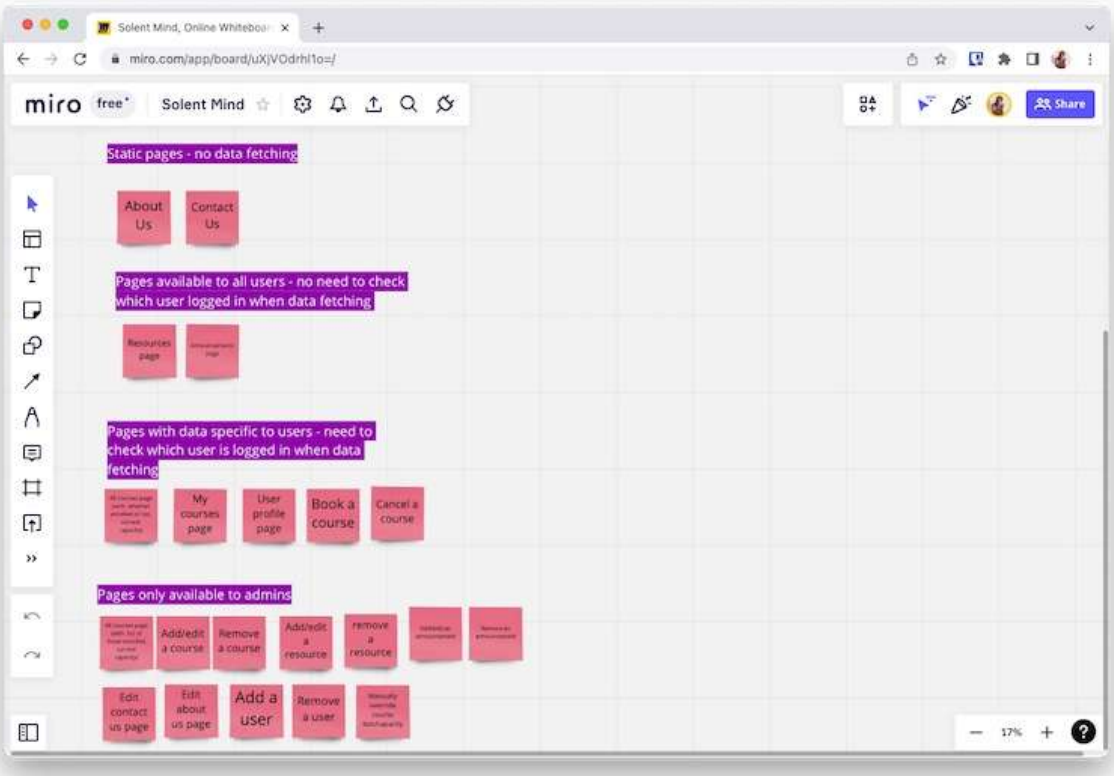
I used Git and GitHub for source control, merging pull requests from the other pair and code reviewing each other's work.



Screenshot of pull request featuring a review comment

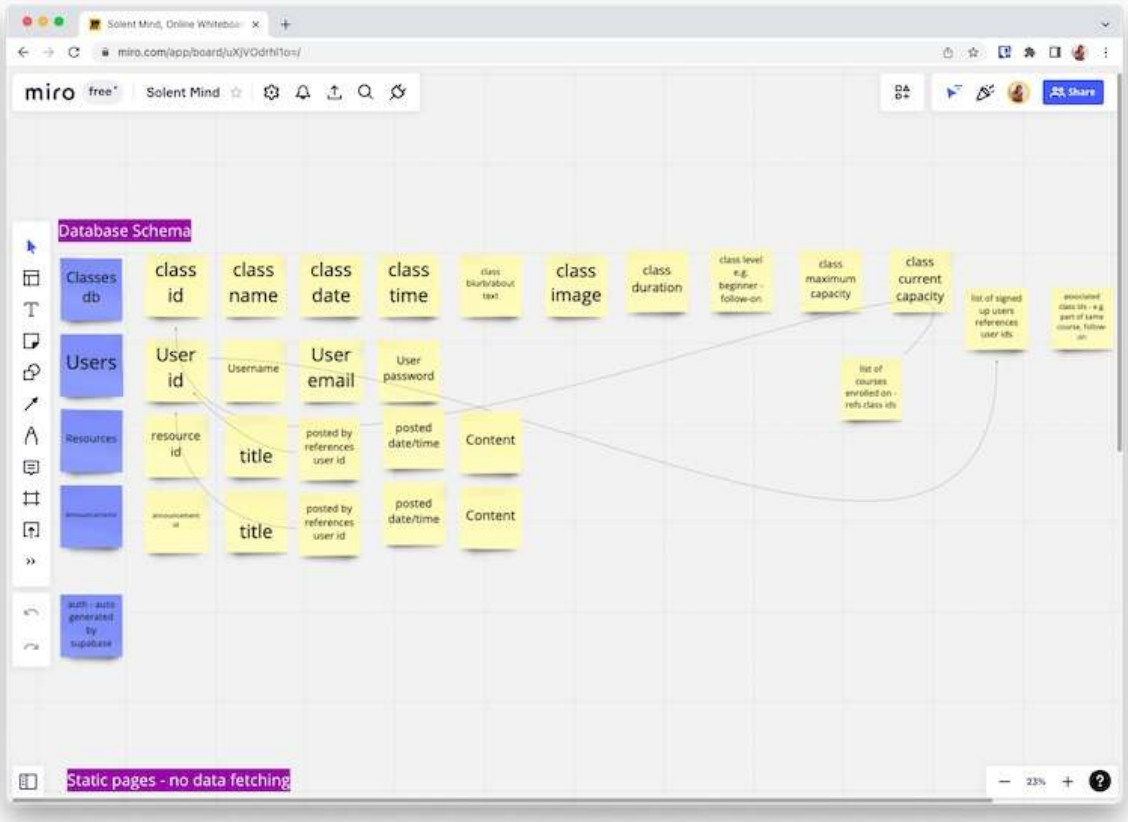
Storing data

I planned which pages would require data and who that data should be visible to on Miro.



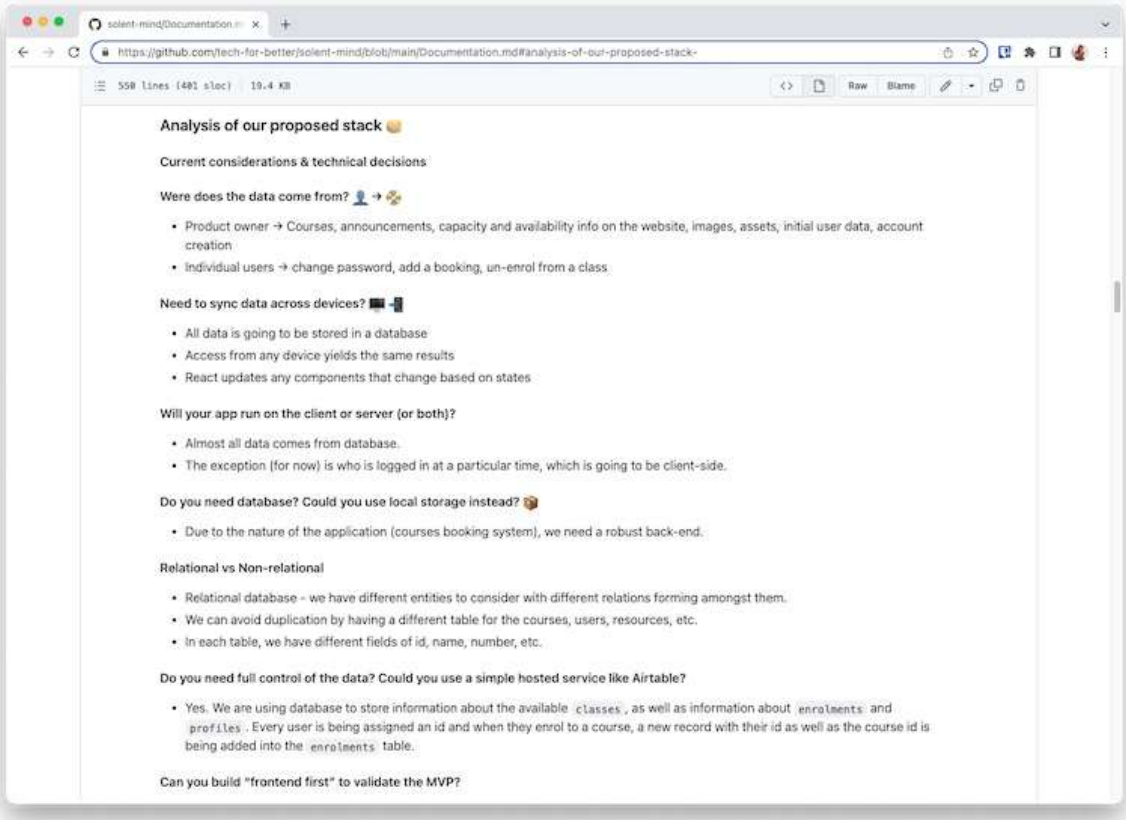
A table detailing which pages will be viewable to which users. Each section defines whether data is required from the DB or not

I used Miro for planning the database schema.



Database schema planned on Miro

And considered the implications of how I'd manage and store data, which I documented in my project report.



550 lines (401 sloc) 19.4 KB

Raw Blame

Analysis of our proposed stack

Current considerations & technical decisions

Were does the data come from? →

- Product owner → Courses, announcements, capacity and availability info on the website, images, assets, initial user data, account creation
- Individual users → change password, add a booking, un-enrol from a class

Need to sync data across devices?

- All data is going to be stored in a database
- Access from any device yields the same results
- React updates any components that change based on states

Will your app run on the client or server (or both)?

- Almost all data comes from database.
- The exception (for now) is who is logged in at a particular time, which is going to be client-side.

Do you need database? Could you use local storage instead?

- Due to the nature of the application (courses booking system), we need a robust back-end.

Relational vs Non-relational

- Relational database - we have different entities to consider with different relations forming amongst them.
- We can avoid duplication by having a different table for the courses, users, resources, etc.
- In each table, we have different fields of id, name, number, etc.

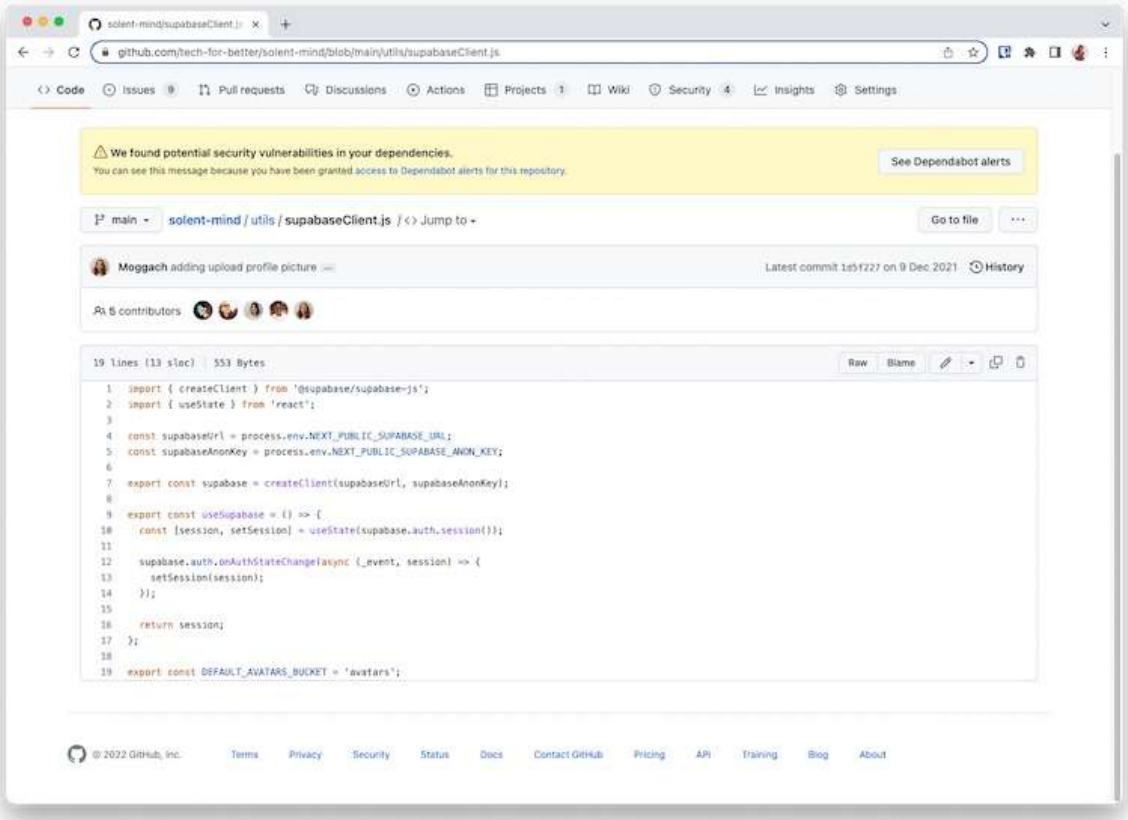
Do you need full control of the data? Could you use a simple hosted service like Airtable?

- Yes. We are using database to store information about the available `classes`, as well as information about `enrolments` and `profiles`. Every user is being assigned an id and when they enrol to a course, a new record with their id as well as the course id is being added into the `enrolments` table.

Can you build "frontend first" to validate the MVP?

Section of the project report which describes data considerations

The database was connected using the Supabase plugin for React. You can see the implementation below:



Supabase client imported into React

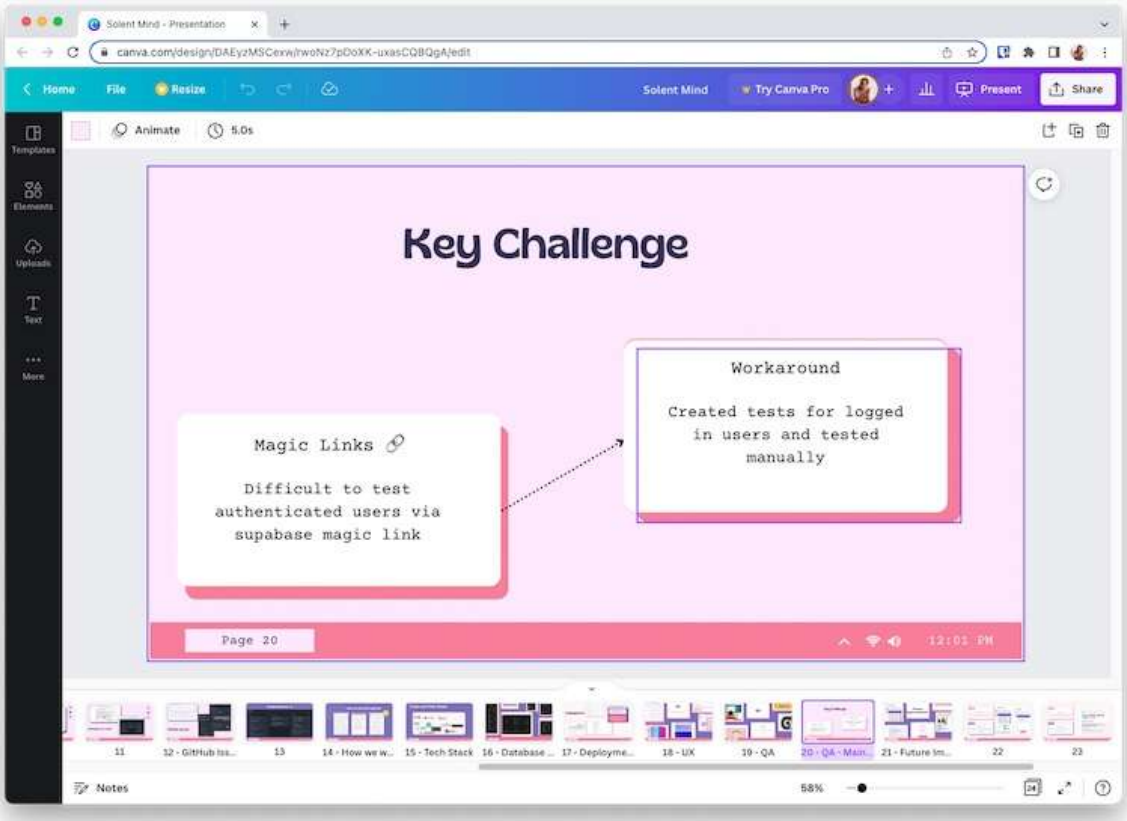
Documenting and Presenting

I documented the project on GitHub across a project report and README. Both of these documents are written with a technical audience in mind.

I also completed non-technical documentation for the Product Owner. They can use this documentation to maintain the app moving forward.

Finally, I presented the product at a showcase event. The audience here were a mixture of technical and non-technical people. My presentation included details in layman's terms as well as some analysis of my technical choices such as the Tech Stack.

A slide from the presentation shows a layman's introduction to a technical problem we faced


Solent Mind - Presentation
canva.com/design/DAEyzMSCexw/rwoNz7pDoXK-uxasCQBQgA/edit
Home
File
Resize
Solent Mind
Try Canva Pro
Present
Share
Animate
5.0s
Templates
Elements
Uploads
Text
More
Key Challenge
Workaround
Created tests for logged in users and tested manually
Magic Links
Difficult to test authenticated users via supabase magic link
Page 20
12:01 PM
11
12 - GitHub Iss...
13
14 - How we w...
15 - Tech Stack
16 - Database ...
17 - Deployme...
18 - UX
19 - QA
20 - QA - Main...
21 - Future Im...
22
23
Notes
58%